

INE5408-03208A/B (20252) - Estruturas de Dados

[Painel](#) / [Cursos](#) / [INE5408-03208A/B \(20252\)](#) / Prova Prática II / [Prova Prática II](#)

 Descrição

 [Visualizar envios](#)

 **Disponível a partir de:** quarta-feira, 26 nov. 2025, 15:10

 **Data de entrega:** sexta-feira, 28 nov. 2025, 17:45

 **Arquivos requeridos:** binary_tree.h, array_list.h ( [Baixar](#))

 **Número máximo de arquivos:** 100

Tipo de trabalho:  Trabalho em grupo [Grupo 14](#)

Enunciado. Implemente 4 (quatro) novos métodos de Árvore Binária de Busca (não balanceada) descritos a seguir. As operações básicas de árvore e lista estão previamente implementadas. A inserção e a remoção na árvore foram modificadas para que cada nó tenha um ponteiro para seu pai/mãe (`parent_`). Acrescente qualquer método auxiliar de nó ou de árvore que considere necessário.

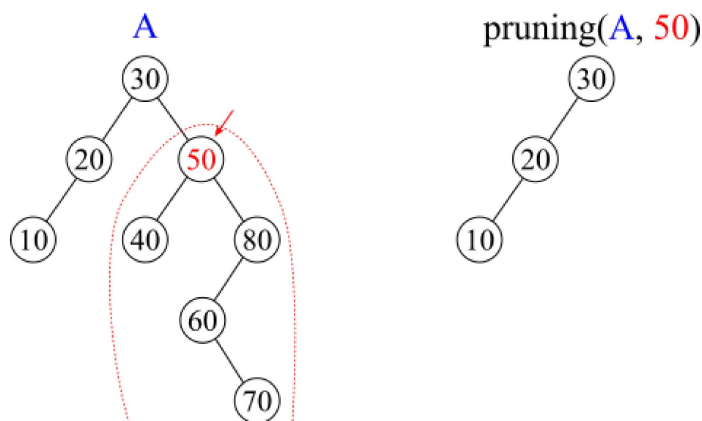
(1) Dada uma árvore binária de busca (o próprio objeto), e um dado qualquer `data` (de tipo `T`), faça sua poda (*pruning*), removendo a subárvore completa enraizada em `data`. Se `data` não existir, a árvore não deve ser alterada ao final do processamento:

```
void pruning(T data);
```

Dica:

- Há um método denominado `acha()` que, ao receber um dado, devolve um ponteiro para o nó que o contém.

Exemplo:



(2) Dadas duas árvores binárias de busca, `A` e `B` (a primeira sendo o próprio objeto; e a segunda, passada por ponteiro, sem cópia), construa uma árvore de saída, a partir de todos os valores das duas primeiras (considere que não há repetições). A árvore binária de busca retornada deve ter a menor altura possível (conforme a figura abaixo):

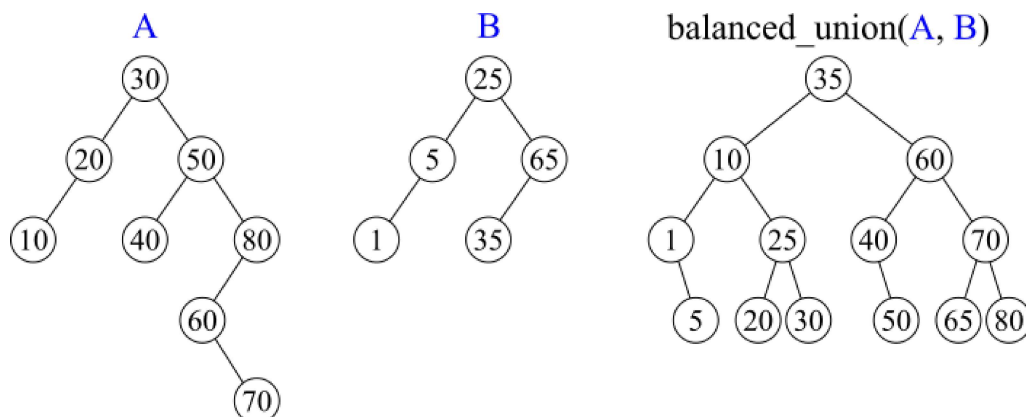
```
BinaryTree<T> balanced_union(BinaryTree<T> *B);
```

Dica:

- Há um método denominado `busca_binaria()` que, ao receber uma árvore vazia e uma lista com valores ordenados, além de índices inicial e final, preenche a árvore com o seguinte algoritmo:
 - Dado uma lista com valores ordenados:
 - inserir a mediana, ou seja, o valor em
 - $\text{índice_central} = (\text{índice_inicial} + \text{índice_final}) \div 2$
 - executar recursivamente para os valores da lista entre os índices:

- `índice_inicial`
- `índice_central - 1`
- executar recursivamente para os valores da lista entre os índices:
 - `índice_central + 1`
 - `índice_final`

Exemplo:



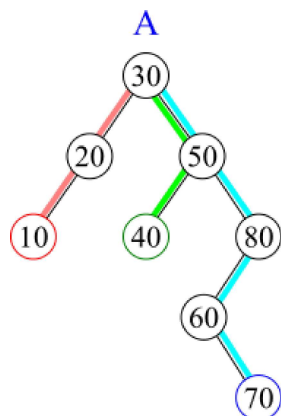
(3) Dada uma árvore binária de busca, `A` (o próprio objeto), construa uma lista de inteiros de saída contendo o comprimento de caminho (quantidade de arestas no caminho) da raiz a cada uma das folhas. Considere a sequência de folhas do percurso "em ordem":

```
ArrayList<int> leaves_path_length();
```

Dica:

- Há um método denominado `acha_folhas()` que, ao receber uma lista vazia de ponteiros para nós, preenche a mesma com ponteiros para cada folha (no percurso "em ordem").

Exemplo:



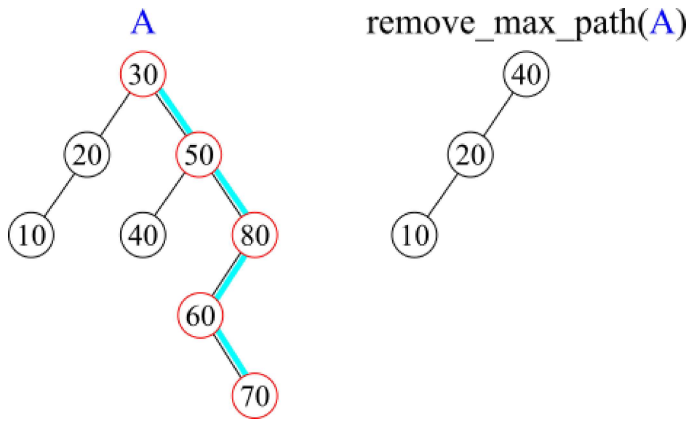
Para esta árvore `A`, as folhas, na sequência de visitação "em ordem", são `10`, `40` e `70`, e a saída deve ter os seguintes comprimentos de caminho:

```
[2, 2, 4]
```

(4) Dada uma árvore binária de busca, `A` (o próprio objeto), construa uma lista de saída (de tipo `T`) contendo os dados/valores, na sequência da folha até a raiz, que compõem o maior comprimento de caminho existente. Essa mesma sequência dados/valores deve ser removida da árvore `A`.

```
ArrayList<T> remove_max_path();
```

Exemplo:



Após a execução, a árvore **A** passa a ser ter apenas três nós (10, 20 e 40) e a lista de saída deve ter a seguinte sequência (da folha para a raiz):

[70, 60, 80, 50, 30]

Arquivos requeridos

binary_tree.h

```

1  ///

```

```

104     } else if (data > data_ && right_ != nullptr) {
105         right_ = remove(data, right_, deleted);
106     } else {
107         if (right_ != nullptr && left_ != nullptr) {
108             Node* temp = right_->minimum();
109             data_ = temp->data_;
110             right_ = remove(data_, right_, deleted);
111         } else if (right_ != nullptr) {
112             Node* temp = right_;
113             data_ = temp->data_;
114             right_ = temp->right_;
115             left_ = temp->left_;
116             right_->parent_ = this;
117             left_->parent_ = this;
118             temp->right_ = temp->left_ = nullptr;
119             delete temp;
120             deleted = true;
121         } else if (left_ != nullptr) {
122             Node* temp = left_;
123             data_ = temp->data_;
124             right_ = temp->right_;
125             left_ = temp->left_;
126             right_->parent_ = this;
127             left_->parent_ = this;
128             temp->right_ = temp->left_ = nullptr;
129             delete temp;
130             deleted = true;
131         }
132     }
133     return deleted;
134 }
135
136 //! Contém
137 bool contains(const T& data) const {
138     if (data < data_)
139         return left_ == nullptr? false : left_->contains(data);
140     else if (data > data_)
141         return right_ == nullptr? false : right_->contains(data);
142     else
143         return true;
144 }
145
146 //! Pré ordem
147 void pre_order(ArrayList<T>& v) const {
148     v.push_back(data_);
149     if (left_ != nullptr)
150         left_->pre_order(v);
151     if (right_ != nullptr)
152         right_->pre_order(v);
153 }
154
155 //! Em ordem
156 void in_order(ArrayList<T>& v) const {
157     if (left_ != nullptr)
158         left_->in_order(v);
159     v.push_back(data_);
160     if (right_ != nullptr)
161         right_->in_order(v);
162 }
163
164 //! Pós ordem
165 void post_order(ArrayList<T>& v) const {
166     if (left_ != nullptr)
167         left_->post_order(v);
168     if (right_ != nullptr)
169         right_->post_order(v);
170     v.push_back(data_);
171 }
172
173
174 T data() {
175     return data_;
176 }
177
178 Node* min() {
179     if (left_ == nullptr)
180         return this;
181     return left_->min();
182 }
183
184 Node* max() {
185     if (right_ == nullptr)
186         return this;
187     return right_->max();
188 }
189
190
191 //! Contém
192 Node *acha(const T& data) {
193     if (data < data_)
194         return left_ == nullptr? nullptr : left_->acha(data);
195     else if (data > data_)
196         return right_ == nullptr? nullptr : right_->acha(data);
197     else
198         return this;
199 }
200
201 //! Pós ordem
202 void percorre(ArrayList<T>& v) {
203     if (left_ != nullptr)
204         left_->percorre(v);
205     if (right_ != nullptr)
206         right_->percorre(v);
207 }

```

```

207         v.push_back(data_);
208     }
209
210     //! Em ordem
211     void acha_folhas(ArrayList<Node *> &v) {
212         if (left_ != nullptr)
213             left_->acha_folhas(v);
214         if (left_ == nullptr && right_ == nullptr) {
215             v.push_back(this);
216         }
217         if (right_ != nullptr)
218             right_->acha_folhas(v);
219     }
220
221
222
223     private:
224     //! Complemento da função de remoção
225     Node* remove(const T& data, Node* arv, bool& deleted) {
226         deleted = false;
227         if (arv == nullptr)
228             return arv;
229         if (data < arv->data_) {
230             arv->left_ = remove(data, arv->left_, deleted);
231             return arv;
232         }
233         if (data > arv->data_) {
234             arv->right_ = remove(data, arv->right_, deleted);
235             return arv;
236         }
237         // dois filhos
238         if (arv->right_ != nullptr && arv->left_ != nullptr) {
239             Node* temp = arv->right_->minimum();
240             arv->data_ = temp->data_;
241             arv->right_ = remove(temp->data_, arv->right_, deleted);
242             return arv;
243         }
244         // um filho ou folha
245         Node* temp = nullptr;
246         if (arv->right_ != nullptr) {
247             temp = arv->right_;
248             if (arv->parent_ == nullptr) {
249                 temp->parent_ = nullptr;
250             }
251         } else {
252             temp = arv->left_;
253             if (arv->parent_ == nullptr) {
254                 temp->parent_ = nullptr;
255             }
256         }
257
258         arv->right_ = nullptr;
259         arv->left_ = nullptr;
260         delete arv;
261         deleted = true;
262         return temp;
263     }
264
265     //! Encontrar o menor
266     Node* minimum() {
267         if (left_ == nullptr)
268             return this;
269         return left_->minimum();
270     }
271 };
272
273 Node* root_{nullptr};
274 std::size_t size_{0};
275 };
276
277
278 template<typename T>
279 void busca_binaria(BinaryTree<T>& A, ArrayList<T>& L,
280                  std::size_t indice_inicial, std::size_t indice_final) {
281     int indice_central;
282     if (indice_inicial >= 0 && indice_final < L.size() &&
283         indice_inicial <= indice_final) {
284         indice_central = (indice_inicial + indice_final) / 2;
285         A.insert(L[indice_central]);
286         busca_binaria(A, L, indice_inicial, indice_central-1);
287         busca_binaria(A, L, indice_central+1, indice_final);
288     }
289 }
290
291
292
293 //*****
294 // Prova prática - implementações necessárias:
295
296 // (1) poda da árvore enraizada em 'data':
297 template<typename T>
298 void BinaryTree<T>::pruning(T data) {
299     //
300     // COLOQUE SEU CÓDIGO AQUI...
301     //
302 }
303
304 // (2) criação de uma árvore binária de busca balanceada da união
305 // da própria com outra B passada por parâmetro:
306 template<typename T>
307 BinaryTree<T>::balanced_union(BinaryTree<T> *B) {
308     BinaryTree<T> t;
309     //
310     // COLOQUE SEU CÓDIGO AQUI

```

```

310 // COLOQUE SEU CÓDIGO AQUI...
311 //
312 return t;
313 }
314
315 // (3) criação de uma lista contendo o comprimento de caminho de
316 // cada folha até a raiz (na sequência "em ordem" de folhas):
317 template<typename T>
318 ArrayList<int> BinaryTree<T>::leaves_path_length() {
319     ArrayList<T> a;
320     //
321     // COLOQUE SEU CÓDIGO AQUI...
322     //
323     return a;
324 }
325
326 // (4) remoção dos nós, da própria árvore, pertencentes ao maior
327 // comprimento de caminho, e retorno da lista de valores da
328 // folha até a raiz:
329 template<typename T>
330 ArrayList<T> BinaryTree<T>::remove_max_path() {
331     ArrayList<T> a;
332     //
333     // COLOQUE SEU CÓDIGO AQUI...
334     //
335     return a;
336 }
337
338 //*****
339
340
341
342 //! Construtor
343 template<typename T>
344 BinaryTree<T>::BinaryTree()
345 {}
346
347 //! Destrutor
348 template<typename T>
349 BinaryTree<T>::~BinaryTree() {
350     delete root_;
351 }
352
353 //! Inserção
354 template<typename T>
355 void BinaryTree<T>::insert(const T& data) {
356     if (empty()) {
357         root_ = new Node(data);
358         if (root_ == nullptr)
359             throw std::out_of_range("Full tree!");
360     } else {
361         root_->insert(data);
362     }
363     ++size_;
364 }
365
366 // Remoção
367 template<typename T>
368 void BinaryTree<T>::remove(const T& data) {
369     if (empty())
370         throw std::out_of_range("Empty tree");
371
372     if (size() != 1u) {
373         if (root_->remove(data))
374             --size_;
375     } else {
376         if (root_->data == data) {
377             delete root_;
378             root_ = nullptr;
379             --size_;
380         }
381     }
382 }
383
384 //! Contém
385 template<typename T>
386 bool BinaryTree<T>::contains(const T& data) const {
387     if (empty())
388         return false;
389     return root_->contains(data);
390 }
391
392 //! Vazio
393 template<typename T>
394 bool BinaryTree<T>::empty() const {
395     return size() == 0;
396 }
397
398 //! Tamanho
399 template<typename T>
400 std::size_t BinaryTree<T>::size() const {
401     return size_;
402 }
403
404 //! Pré ordem
405 template<typename T>
406 ArrayList<T> BinaryTree<T>::pre_order() const {
407     ArrayList<T> v{size_};
408     if (!empty())
409         root_->pre_order(v);
410     return v;
411 }
412
413 //! Em ordem

```

```
414     template<typename T>
415     ArrayList<T> BinaryTree<T>::in_order() const {
416         ArrayList<T> v{size_};
417         if (!empty())
418             root_->in_order(v);
419         return v;
420     }
421
422     /// Pós ordem
423     template<typename T>
424     ArrayList<T> BinaryTree<T>::post_order() const {
425         ArrayList<T> v{size_};
426         if (!empty())
427             root_->post_order(v);
428         return v;
429     }
430 }
431 // namespace structures
432
433 #endif
434
```

array_list.h

```

1  ///  
2  
3  #ifndef STRUCTURES_ARRAY_LIST_H  
4  #define STRUCTURES_ARRAY_LIST_H  
5  
6  #include <stdint>  
7  #include <stdexcept> // C++ exceptions  
8  
9  namespace structures {  
10  
11  ///  
12  template<typename T>  
13  class ArrayList {  
14  public:  
15      ///  
16      ArrayList();  
17      ///  
18      explicit ArrayList(std::size_t max_size);  
19      ///  
20      ~ArrayList();  
21  
22      ///  
23      void clear();  
24      ///  
25      void push_back(const T& data);  
26      ///  
27      void push_front(const T& data);  
28      ///  
29      void insert(const T& data, std::size_t index);  
30      ///  
31      void insert_sorted(const T& data);  
32      ///  
33      T pop(std::size_t index);  
34      ///  
35      T pop_back();  
36      ///  
37      T pop_front();  
38      ///  
39      void remove(const T& data);  
40      ///  
41      bool full() const;  
42      ///  
43      bool empty() const;  
44      ///  
45      bool contains(const T& data) const;  
46      ///  
47      std::size_t find(const T& data) const;  
48      ///  
49      std::size_t size() const;  
50      ///  
51      std::size_t max_size() const;  
52      ///  
53      T& at(std::size_t index);  
54      ///  
55      T& operator[](std::size_t index);  
56      ///  
57      const T& at(std::size_t index) const;  
58      ///  
59      const T& operator[](std::size_t index) const;  
60  
61  private:  
62      T* contents;  
63      std::size_t size_;  
64      std::size_t max_size_;  
65  
66      static const auto DEFAULT_MAX = 50u;  
67  };  
68  
69  } // namespace structures  
70  
71  ///  
72  //-----  
73  
74  template<typename T>  
75  structures::ArrayList<T>::ArrayList() {  
76      max_size_ = DEFAULT_MAX;  
77      contents = new T[max_size_];  
78      size_ = 0;  
79  }  
80  
81  
82  template<typename T>  
83  structures::ArrayList<T>::ArrayList(std::size_t max_size) {  
84      max_size_ = max_size;  
85      contents = new T[max_size_];  
86      size_ = 0;  
87  }  
88  
89  template<typename T>  
90  structures::ArrayList<T>::~~ArrayList() {  
91      delete []contents;  
92  }  
93  
94  template<typename T>  
95  void structures::ArrayList<T>::clear() {  
96      size_ = 0;  
97  }  
98  
99  template<typename T>  
100 void structures::ArrayList<T>::push_back(const T& data) {  
101     insert(data, size_);  
102 }  
103

```

```

104 template<typename T>
105 void structures::ArrayList<T>::push_front(const T& data) {
106     insert(data, 0);
107 }
108
109 template<typename T>
110 void structures::ArrayList<T>::insert(const T& data, std::size_t index) {
111     if (full()) {
112         throw std::out_of_range("full list");
113     } else if (index > size()) {
114         throw std::out_of_range("invalid index");
115     }
116     if (size() > 0) {
117         for (std::size_t i = size(); i > index; i--) {
118             contents[i] = contents[i-1];
119         }
120     }
121     contents[index] = data;
122     size_++;
123 }
124
125 template<typename T>
126 void structures::ArrayList<T>::insert_sorted(const T& data) {
127     if (full()) {
128         throw std::out_of_range("full list");
129     }
130     std::size_t index = 0;
131     while (index < size() && contents[index] < data) {
132         index++;
133     }
134     insert(data, index);
135 }
136
137 template<typename T>
138 T structures::ArrayList<T>::pop(std::size_t index) {
139     if (empty()) {
140         throw std::out_of_range("empty list");
141     } else if (index >= size()) {
142         throw std::out_of_range("invalid index");
143     }
144     T data = contents[index];
145     for (std::size_t i = index; i < size()-1; i++) {
146         contents[i] = contents[i+1];
147     }
148     size_--;
149     return data;
150 }
151
152 template<typename T>
153 T structures::ArrayList<T>::pop_back() {
154     if (empty()) {
155         throw std::out_of_range("empty list");
156     }
157     return pop(size()-1);
158 }
159
160 template<typename T>
161 T structures::ArrayList<T>::pop_front() {
162     if (empty()) {
163         throw std::out_of_range("empty list");
164     }
165     return pop(0);
166 }
167
168 template<typename T>
169 void structures::ArrayList<T>::remove(const T& data) {
170     if (empty()) {
171         throw std::out_of_range("empty list");
172     }
173     std::size_t index;
174     for (index = 0; index < size(); index++) {
175         if (contents[index] == data)
176             break;
177     }
178     if (index == size()) {
179         throw std::out_of_range("data not found");
180     }
181     pop(index);
182 }
183
184 template<typename T>
185 bool structures::ArrayList<T>::empty() const {
186     if (size() == 0)
187         return true;
188     return false;
189 }
190
191 template<typename T>
192 bool structures::ArrayList<T>::full() const {
193     return (size() == max_size());
194 }
195
196 template<typename T>
197 bool structures::ArrayList<T>::contains(const T& data) const {
198     size_t index = find(data);
199     return (index < size());
200 }
201
202 template<typename T>
203 std::size_t structures::ArrayList<T>::find(const T& data) const {
204     if (empty()) {
205         throw std::out_of_range("empty list");
206     }
207     for (std::size_t i = 0; i < size(); i++) {
208         if (contents[i] == data)
209             return i;
210     }
211     return size();
212 }

```

```
207     std::size_t index;
208     for (index = 0; index < size(); index++) {
209         if (contents[index] == data)
210             return index;
211     }
212     return size(); // not found
213 }
214
215 template<typename T>
216 std::size_t structures::ArrayList<T>::size() const {
217     return size_;
218 }
219
220 template<typename T>
221 std::size_t structures::ArrayList<T>::max_size() const {
222     return max_size_;
223 }
224
225 template<typename T>
226 T& structures::ArrayList<T>::at(std::size_t index) {
227     if (empty()) {
228         throw std::out_of_range("empty list");
229     } else if (index >= size()) {
230         throw std::out_of_range("invalid index");
231     }
232     return contents[index];
233 }
234
235 template<typename T>
236 T& structures::ArrayList<T>::operator[](std::size_t index) {
237     return at(index);
238 }
239
240 template<typename T>
241 const T& structures::ArrayList<T>::at(std::size_t index) const {
242     const T aux = at(index);
243     return aux;
244 }
245
246 template<typename T>
247 const T& structures::ArrayList<T>::operator[](std::size_t index) const {
248     return at(index);
249 }
250
251 #endif
252
```

✉ Contate o suporte do site 

[VPL](#)

Você acessou como Elisa Belquis de Assumpcao (24202220) (Sair)

INE5408-03208A/B (20252)

Moodle UFSC - Presencial